

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

- 01 Basic Data Structures
- 02
- 03
- 04
- 05

Basic Data Structures

- A data structure is a group of data elements that are put together under one name
- It defines a particular way of storing and organizing data in a computer for efficient use

Lists

- List is a versatile data type available in Python
- It is a sequence in which elements are written as a list of comma-separated values (items) between [] brackets
- List can have elements of different data types

Creating Lists Examples

Empty List

```
L = []
```

List of Numbers

```
list_A = [1,2,3,4,5]
```

```
Print(list_A)
```

```
[1, 2, 3, 4, 5]
```

Creating Lists Examples

List of Characters

```
list_B = ['A' , 'b' , 'C' , 'd' , 'E']
```

```
Print(list_B)
```

```
['A', 'b', 'C', 'd', 'E']
```

Creating Lists Examples

List of Strings

```
list_C = ["Good", "Going"]
```

```
Print(list_C)
```

```
['Good', 'Going']
```

Access Values in Lists

- Similar to strings, lists can also be sliced and concatenated
- To access values in lists, square brackets are used along with the index

`seq = list[start: stop: step]`

Access Values in Lists

`seq1 = list[::2]` #get every other element starting with index 0

`seq2 = list[1::2]` #get every other element starting with index 1

Access Values in Lists

```
num_list = [1,2,3,4,5,6,7,8,9,10]
print("num_list is : ", num_list)
print("First element in the list is ", num_list[0])
print("num_list[2:5] = ", num_list[2:5])
print("num_list[::2] = ", num_list[::2])
print("num_list[1::3] = ", num_list[1::3])
```

OUTPUT

```
num_list is :  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
First element in the list is  1
num_list[2:5] =  [3, 4, 5]
num_list[::2] =  [1, 3, 5, 7, 9]
num_list[1::3] =  [2, 5, 8]
```

Updating values in Lists

- Lists can be updated easily
- Append new values or deleting some values

Updating values in Lists

- Once created, one or more elements of a list can be easily updated by giving the slice on the left-hand side of the assignment operator.
- You can also append new values in the list and remove existing value(s) from the list using the *append() method* and *del statement* respectively.

Updating values in Lists

```
num_list = [1,2,3,4,5,6,7,8,9,10]
print("List is : ", num_list)
num_list[5] = 100
print("List after updation is : ", num_list)
num_list.append(200)
print("List after appending a value is ", num_list)
del num_list[3]
print("List after deleting a value is ", num_list)
```

Programming Tip: `append()` and `insert()` methods are list methods. They cannot be called on other values such as strings or integers.

OUTPUT

```
List is :  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
List after updation is :  [1, 2, 3, 4, 5, 100, 7, 8, 9, 10]
List after appending a value is  [1, 2, 3, 4, 5, 100, 7, 8, 9, 10, 200]
List after deleting a value is  [1, 2, 3, 5, 100, 7, 8, 9, 10, 200]
```

Updating values in Lists

```
num_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
del num_list[2:4]    #deletes numbers at index 2 and 3  
print(num_list)
```

Updating values in Lists

```
num_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
del num_list[:]      #deletes all the numbers from the list
```

```
print(num_list)
```

```
del num_list         #deletes the list
```

Nested Lists: List inside another List

```
num_list = [1, 2, 9]
```

```
print("The original list is:", num_list)
```

```
num_list[1] = [3, 5, 7]
```

```
print("After inserting another list:", num_list)
```


Nested Lists

- Nested list means a list within another list

```
list1 = [1, 'a', "abc", [2,3,4,5], 8.9]
i=0
while i<(len(list1)):
    print("List1[",i,"] = ",list1[i])
    i+=1
```

OUTPUT

```
List1[0] = 1
List1[1] = a
List1[2] = abc
List1[3] = [2, 3, 4, 5]
List1[4] = 8.9
```

Nested Lists

- Nested list means a list within another list

```
list = [1, 'a', "abc", [2,3,4,5], 8.9]
```

- You can specify an element in the nested list by using a set of indices

```
print(list[3][1])    #it will print the second element in nested list
```

Nested Lists

```
list=[1, 'a', "abc", [2,3,4,5], 8.9]
```

```
print(list[3][1]) #it will print the second element in nested list
```

- First index specifies starting of the nested list in the main list
- Second index specifies the index of the element within the nested list

Cloning Lists

- If you want to modify a list and also keep a copy of the original list, then you should create a separate copy of the list (not just the reference).
- This process is called *cloning*. The slice operation is used to clone a list.

Cloning Lists Example

```
list1 = [1,2,3,4,5,6,7,8,9,10]
list2 = list1                #copies a list using reference
print("List1 = ", list1)
print("List2 = ", list2)    #both lists point to the same list
list3 = list1[2:6]
print("List3 = ", list3)    #list is a clone of list1
```

OUTPUT

```
List1 =  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
List2 =  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
List3 =  [3, 4, 5, 6]
```

List Operations: length

'len' operation

It returns the length of a list

```
List_num = [1, 2, 3]
```

```
len(List_num)
```

```
3
```

List Operations: Concatenation

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5]
```

```
list3 = list1 + list2
```

```
print(list3)
```

```
[1, 2, 3, 4, 5]
```

List Operations: repetition

```
list1 = [1, 2, 3]
```

```
list2 = list1*2
```

```
print(list2)
```

```
[1, 2, 3, 1, 2, 3]
```


List Operations: in

Checks if the value is in the list

```
list1 = ['a', 'e', 'i', 'o', 'u']
```

```
'a' in list1
```

```
True
```

List Operations: not in

Checks if the value is not in the list

```
list1 = ['a', 'e', 'i', 'o', 'u']
```

```
'u' not in list1
```

```
False
```

List Operations

8.2.5 Basic List Operations

Lists behave in the similar way as strings when operators like + (concatenation) and * (repetition) are used. It works similar in case of operations discussed in Table 8.1.

Table 8.1 Operations on Lists

Operation	Description	Example	Output
len	Returns length of list	<code>len([1,2,3,4,5,6,7,8,9,10])</code>	10
concatenation	Joins two lists	<code>[1,2,3,4,5] + [6,7,8,9,10]</code>	<code>[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]</code>
repetition	Repeats elements in the list	<code>"Hello", "World"*2</code>	<code>['Hello', 'World', 'Hello', 'World']</code>
in	Checks if the value is present in the list	<code>'a' in ['a','e','i','o','u']</code>	True
not in	Checks if the value is not present in the list	<code>3 not in [0,2,4,6,8]</code>	True
max	Returns maximum value in the list	<pre>>>> num_list = [6,3,7,0,1,2,4,9] >>> print(max(num_list))</pre>	9
min	Returns minimum value in the list	<pre>>>> num_list = [6,3,7,0,1,2,4,9] >>> print(min(num_list))</pre>	0
sum	Adds the values in the list that has numbers	<pre>num_list = [1,2,3,4,5,6,7,8,9,10] print("SUM = ", sum(num_list))</pre>	SUM = 55

Contd

List Operations

Table 8.1 Contd

Operation	Description	Example	Output
all	Returns True if all elements of the list are true (or if the list is empty)	<pre>>>> num_list = [0,1,2,3] >>> print(all(num_list))</pre>	False
any	Returns True if any element of the list is true. If the list is empty, returns False	<pre>>>> num_list = [6,3,7,0,1,2,4,9] >>> print(any(num_list))</pre>	True
list	Converts an iterable (tuple, string, set, dictionary) to a list	<pre>>>> list1 = list("HELLO") >>> print(list1)</pre>	['H', 'E', 'L', 'L', 'O']
sorted	Returns a new sorted list. The original list is not sorted.	<pre>>>> list1 = [3,4,1,2,7,8] >>> list2 = sorted(list1) >>> print(list2)</pre>	[1, 2, 3, 4, 7, 8]

List Methods

- Python has various methods to help programmers work efficiently with lists
- Some examples in table

List Operations

Method	Description	Syntax	Example	Output
<code>append()</code>	Appends an element to the list. In <code>insert()</code> , if the index is 0, then element is inserted as the first element and if we write, <code>list.insert(len(list), obj)</code> , then it inserts <code>obj</code> as the last element in the list. That is, if <code>index= len(list)</code> then <code>insert()</code> method behaves exactly same as <code>append()</code> method.	<code>list. append(obj)</code>	<pre>num_list = [6,3,7,0,1,2,4,9] num_list. append(10) print (num_list)</pre>	<pre>[6,3,7, 0,1,2, 4,9,10]</pre>
<code>count()</code>	Counts the number of times an element appears in the list.	<code>list.count (obj)</code>	<pre>print(num_list. count(4))</pre>	<pre>1</pre>
<code>index()</code>	Returns the lowest index of <code>obj</code> in the list. Gives a <code>ValueError</code> if <code>obj</code> is not present in the list.	<code>list. index(obj)</code>	<pre>>>> num_list = [6,3,7,0,3,7,6,0] >>> print(num_ list.index(7))</pre>	<pre>2</pre>

List Operations

<code>insert()</code>	Inserts obj at the specified index in the list.	<code>list. insert(index, obj)</code>	<pre>>>> num_list = [6,3,7, [6,3,7,0,3,7,6,0] >>> num_list. insert(3, 100) >>> print(num_list)</pre>
<code>pop()</code>	Removes the element at the specified index from the list. Index is an optional parameter. If no index is specified, then removes the last object (or element) from the list.	<code>list. pop([index])</code>	<pre>num_list = 9 [6,3,7,0,1,2,4,9] print(num_list. pop()) print(num_list)</pre>

List Methods

<code>remove()</code>	Removes or deletes obj from the list. ValueError is generated if obj is not present in the list. If multiple copies of obj exists in the list, then the first value is deleted.	<code>list. remove(obj)</code>	<pre>>>> num_list = [6,3,7, [6,3,7,0,1,2,4,9] >>> num_list. remove(0) >>> print(num_list)</pre>
<code>reverse()</code>	Reverse the elements in the list.	<code>list. reverse()</code>	<pre>>>> num_list = [9, 4, [6,3,7,0,1,2,4,9] >>> num_list. reverse() >>>print(num_list)</pre>
<code>sort()</code>	Sorts the elements in the list.	<code>list.sort()</code>	<pre>>>> num_list = [9, 4, [6,3,7,0,1,2,4,9] >>> num_list. sort() >>> print(num_list)</pre>
<code>extend()</code>	Adds the elements in a list to the end of another list. Using + or += on a list is similar to using extend().	<code>list1. extend(list2)</code>	<pre>>>> num_list1 = [1, 2, [1,2,3,4,5] >>> num_list2 = [6, 7, 8, [6,7,8,9,10] >>> num_list1. extend(num_list2) >>>print(num_ list1)</pre>

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025